

What is ROS ?

 ROS  2

What is ROS ?

- ROS (Robot Operating System) is a framework (**not** an operating system) developed to make robotics development easier, by using a collection of tools, software libraries, and documentation.
- Its federated package model allows unrelated entities to create and reuse compatible software.
 - Mostly **open source** and **free** to use.



A brief history of ROS

- The ROS initiative started at Stanford University in 2006 with Keenan Wyrobek and Erik Berger, developed in 2007 at the Stanford Artificial Intelligence Laboratory
- After Stanford, ROS was developed until 2014 by Willow Garage
- In 2014, it was taken over by OSRF (Open Source Robotics Foundation)
- ROS 2 was released in 2017, as an answer to the need of the industry to have a suitable version of ROS
- The last ROS 1 release (Noetic Ninjemys) was released in 2020, with support until May, 2025
- Today used by many [robots](#), universities and companies
- De facto standard for robot programming



<https://www.ros.org/news/2014/09/ros-running-on-iss.html>

ROS on ISS !

ROS 2 - overview

- Latest LTS (Long-term support) version: Jazzy Jalisco
- Supported operating systems: **Linux**, Windows, Mac
- Client libraries: rclcpp (C++), rclpy (Python). There are also community maintained ones (Java, Rust)

Distro	Release date	Logo	EOL date
Kilted Kaiju	May 23, 2025		December 2026
Jazzy Jalisco	May 23, 2024		May 2029
Iron Irwini	May 23, 2023		December 4, 2024
Humble Hawksbill	May 23, 2022		May 2027

ROS Philosophy

- **Peer to peer:** Individual programs communicate over defined API (ROS messages, services, etc.)
- **Distributed:** Programs can be run on multiple computers and communicate over the network
- **Multi-lingual:** ROS modules can be written in any language for which a client library exists (C++, Python, MATLAB, Java, etc.)
- **Light-weight:** Stand-alone libraries are wrapped around with a thin ROS layer
- **Free and open-source:** Most ROS software is open-source and free to use

ROS Scope

ROS 2 supports a broad range of robotics applications, from education and research to product development and deployment. It comprises a large set of interrelated software components that are commonly used to develop robotics applications. The software ecosystem is divided into three categories:

Middleware: Referred to as the plumbing, the ROS 2 middleware encompasses communication among components, from network application program interfaces (APIs) to message parsers.

Algorithms: ROS 2 provides many of the algorithms commonly used when building robotics applications, e.g., perception, Simultaneous Localization and Mapping (SLAM), planning, and beyond.

Developer tools: ROS 2 includes a suite of command-line and graphical tools for configuration, launch, introspection, visualization, debugging, simulation, and logging. There is also a large suite of tools for source management, build processes, and distribution.

Why ROS ?



Why ROS ?

- It came as a response to the big issue in robotics research that everyone was reinventing the wheel, and thus not focusing on actual research
- Provides a framework with infrastructure for visualizing, controlling, and simulating robots
- ROS has many packages, some of which include difficult-to-implement yet state-of-the-art algorithms
- Many robots provide ROS interfaces: learn ROS once, be able to use many robots
- ROS is **open source**. No black-boxes: it is possible to understand and modify to fit your needs



<https://spectrum.ieee.org/the-origin-story-of-ros-the-linux-of-robotics>

from the original pitch deck from 2006

Why ROS ?

- It came as a response to the big issue in robotics research that everyone was reinventing the wheel, and thus not focusing on actual research
- Provides a framework with infrastructure for visualizing, controlling, and simulating robots
- ROS has many packages, some of which include difficult-to-implement yet state-of-the-art algorithms
- Many robots provide ROS interfaces: learn ROS once, be able to use many robots
- ROS is **open source**. No black-boxes: it is possible to understand and modify to fit your needs



<https://spectrum.ieee.org/the-origin-story-of-ros-the-linux-of-robotics>

from the original pitch deck from 2006

Main advantages

- Standards and conventions (for communication, package organization, [REPs](#) - ROS Enhancement Proposal)
- Sensors and hardware are plug-and-play, as long as the manufacturers provide ROS drivers
- Distributed system - works on any computer and allows computers in the same network to talk to each other
- Actively developed by big companies (Nvidia, Sony, Canonical) that are backing up a community of experienced developers
- Active open source community
 - [OpenRobotics Discourse - ROS](#)
 - [Robotics Stack Exchange - ROS](#)
- It has slowly managed to become the standard framework to use for custom robotics software development

Is it bad in any way?



Why ROS 2 ?

- ROS 1 was designed for use with a single robot and hence had a central component (roscore) that needed to be accounted for
- ROS 1 has no real-time performance guarantees. ROS 2 is working on enabling real-time control
- ROS 1 used a custom service/message middleware whereas ROS 2 uses industry standard versions.

Category	ROS 1	ROS 2
Network Transport	Bespoke protocol built on TCP/UDP	Existing standard (DDS), with abstraction supporting addition of others
Network Architecture	Central name server (roscore)	Peer-to-peer discovery
Platform Support	Linux	Linux, Windows, macOS
Client Libraries	Written independently in each language	Sharing a common underlying C library (rcl)
Node vs. Process	Single node per process	Multiple nodes per process
Threading Model	Callback queues and handlers	Swappable executor
Node State Management	None	Lifecycle nodes
Embedded Systems	Minimal experimental support (rosserial)	Commercially supported implementation (micro-ROS)
Parameter Access	Auxilliary protocol built on XMLRPC	Implemented using service calls
Parameter Types	Type inferred when assigned	Type declared and enforced

How ROS ?



ROS Usage

<https://www.youtube.com/watch?v=EbzfiNpLzvg>

<https://www.youtube.com/watch?v=xqjVTE7QvOg>

<https://www.youtube.com/shorts/D6YymP-CSj0?feature=share>

https://www.youtube.com/watch?v=R_5HW1TUDQk

<https://youtu.be/509jvX-1o1A?t=106>

<https://www.youtube.com/watch?v=aC8vivERCvg>

<https://youtu.be/HDXr6Pwu9fM?t=91>

Thank You...

...for your attention



First, install !

[WSL 2 + CoppeliaSim + ROS 2](#) - tutorial video

[ROS 2 Documentation \(Ubuntu deb packages\)](#) - official documentation